

MODULE 3 WORKBOOK

BUILDING YOUR TOOL

From Idea to Working MVP in 14 Days

"The way to get started is to quit talking and begin doing."

- Walt Disney

This workbook is your build companion. It contains practical templates, decision frameworks, code snippets, and tracking tools to guide you from first commit to launch day. Work through each section as you progress through Module 3's lessons.

Student Name:

Tool Name: _____

Build Start Date:

Target Launch Date:

MODULE 3: BUILDING

OVERVIEW & SETUP

Module Progress Checklist

***Instructions:** Check each box as you complete the corresponding lesson. This helps you track progress and ensures you don't skip critical steps.*

- ☐ **Lesson 1: Tech Stack Selection** - Choose your technologies and justify decisions
- ☐ **Lesson 2: MVP Architecture Planning** - Map out your application structure
- ☐ **Lesson 3: Building Core Features** - Implement your main functionality
- ☐ **Lesson 4: UI/UX Best Practices** - Design an intuitive interface
- ☐ **Lesson 5: Testing & Bug Fixing** - Ensure quality before launch
- ☐ **Lesson 6: Launch Day Preparation** - Deploy and go live!

QUICK WIN GENERATOR

Start building momentum TODAY:

Next 60 Minutes: Set up your development environment and push your first "Hello World" commit to GitHub. Seeing code live, even if it's just "Hello World," builds psychological momentum.

■ Environment ready and first commit done!

Why this matters: Many developers spend weeks "preparing" and never actually build. Pushing code immediately breaks analysis paralysis.

MODULE 3: BUILDING

OVERVIEW & SETUP

Module Progress Checklist

***Instructions:** Check each box as you complete the corresponding lesson. This helps you track progress and ensures you don't skip critical steps.*

- ☐ **Lesson 1: Tech Stack Selection** - Choose your technologies and justify decisions
- ☐ **Lesson 2: MVP Architecture Planning** - Map out your application structure
- ☐ **Lesson 3: Building Core Features** - Implement your main functionality
- ☐ **Lesson 4: UI/UX Best Practices** - Design an intuitive interface
- ☐ **Lesson 5: Testing & Bug Fixing** - Ensure quality before launch
- ☐ **Lesson 6: Launch Day Preparation** - Deploy and go live!

QUICK WIN GENERATOR

Start building momentum TODAY:

Next 60 Minutes: Set up your development environment and push your first "Hello World" commit to GitHub. Seeing code live, even if it's just "Hello World," builds psychological momentum.

■ Environment ready and first commit done!

Why this matters: Many developers spend weeks "preparing" and never actually build. Pushing code immediately breaks analysis paralysis.

Pre-Build Assessment

Honest self-assessment helps set realistic expectations. There are no wrong answers—this establishes your baseline.

My technical confidence level (1-10): _____

1-3: Beginner (follow No-Code path) | 4-6: Some experience | 7-10: Confident coder

Hours per day I can dedicate to building: _____

1-2 hours: 3-week build | 3-4 hours: 2-week build | 5+ hours: 1-week build

My biggest technical challenge will be:

Example: "I'm worried about connecting the frontend to the database. I've never used APIs before and don't understand authentication flows."

MODULE 3: BUILDING

TECH STACK SELECTION

Tech Stack Decision Tree

How to use this page: Review both options, check the technologies you'll use, then complete the final stack decision table. Your choices here determine your entire development approach.

PATH DECISION (CHOOSE ONE)

MODULE 3: BUILDING

TECH STACK SELECTION

Tech Stack Decision Tree

***How to use this page:** Review both options, check the technologies you'll use, then complete the final stack decision table. Your choices here determine your entire development approach.*

PATH DECISION (CHOOSE ONE)

PATH A: NO-CODE

Best for business-first founders

Timeline: 3-5 days

Cost: \$29-99/month

PATH B: LOW-CODE

Best for technical tinkerers

Timeline: 5-7 days

Cost: Near-zero (free tiers)

FRONTEND SELECTION

Option A: Simple Stack

- ☐ HTML/CSS/Vanilla JS
- ☐ Bootstrap/Tailwind CSS
- ☐ jQuery (if needed)

Best for: Conversion tools, calculators, generators

Examples: Password Generator, Invoice Creator, QR Code Maker

Option B: Modern Stack

- ☐ React/Vue/Svelte
- ☐ Next.js/Nuxt
- ☐ TypeScript

Best for: Complex interactions, dashboards, SaaS platforms

Examples: Project management tools, CRM systems, analytics dashboards

BACKEND SELECTION

Option A: Serverless (RECOMMENDED)

- ☐ Supabase (PostgreSQL + Auth)
- ☐ Vercel/Netlify Functions
- ☐ Firebase (Google)

Best for: Low maintenance, auto-scaling, generous free tiers

Free tier: 500MB DB, 50K users, 1GB storage

Option B: Traditional Backend

- ☐ Node.js/Express
- ☐ Python/Django/Flask
- ☐ Ruby on Rails / PHP

Best for: Complex business logic, full control, existing expertise

Requires: Server management, deployment config, ongoing maintenance

AI Integration Options

- If your tool uses AI capabilities, add:
- ☐ OpenAI API (GPT-4, DALL-E, Whisper)
 - ☐ Anthropic Claude API (Advanced reasoning)
 - ☐ Stability AI (Image generation)
- Cost: ~\$0.002-0.03 per API call (pay as you go)

MY FINAL STACK DECISION

COMPONENT	TECHNOLOGY CHOICE	WHY THIS CHOICE?
-----------	-------------------	------------------

Frontend Framework		
Backend/API		
Database		
Hosting		
Payment Processing		
AI Services (if any)		

Example Decision:

Frontend: HTML/CSS/JS + Tailwind → "Simple tool, no complex state management needed"

Backend: Supabase → "Free tier handles 50K users, includes auth out of box"

Hosting: Vercel → "Free SSL, auto-deploys from GitHub, zero config"

Payment: Stripe → "Industry standard, excellent docs, easy integration"

AI: OpenAI GPT-3.5 → "Need text generation, \$0.002/1K tokens is affordable"

MODULE 3: BUILDING

MVP FEATURES

MVP Feature Checklist

Critical MVP Rule: If a feature takes more than 8 hours to build, it's NOT an MVP feature. Your MVP should be launchable in 7-14 days maximum. Be ruthless about cutting scope.

MODULE 3: BUILDING

MVP FEATURES

MVP Feature Checklist

Critical MVP Rule: *If a feature takes more than 8 hours to build, it's NOT an MVP feature. Your MVP should be launchable in 7-14 days maximum. Be ruthless about cutting scope.*

CORE FEATURES (MUST HAVE FOR LAUNCH)

Feature Definition Framework: Each feature should have a clear user story: "As a [user type], I want to [action], so that [benefit]."

☐ **Feature 1:** _____

User story: As a _____, I want to _____, so that _____

Estimated time: _____ hours | Actual: _____ hours

☐ **Feature 2:** _____

User story: As a _____, I want to _____, so that _____

Estimated time: _____ hours | Actual: _____ hours

☐ **Feature 3:** _____

User story: As a _____, I want to _____, so that _____

Estimated time: _____ hours | Actual: _____ hours

Example for Password Generator MVP:

Feature 1: Generate secure random passwords

User story: As a user, I want to generate random passwords with customizable length and character types, so that I can create secure credentials quickly.

Estimated time: 4 hours | Actual: 5 hours

INFRASTRUCTURE REQUIREMENTS

These are technical foundations. Check each as you implement them. Don't skip security items—they're non-negotiable.

- ☐ User authentication system (signup/login/logout)

Tool: Supabase Auth | Time: 2-3 hours | Status: Essential if you need users

- ☐ Database schema designed and implemented

Tool: Supabase/PostgreSQL | Time: 1-2 hours | Status: Essential for data persistence

- ☐ Payment integration (Stripe or Lemon Squeezy)

Tool: Stripe Checkout | Time: 3-4 hours | Status: Required for monetization

- ☐ Email notifications (welcome, receipts, alerts)

Tool: Resend/SendGrid | Time: 2 hours | Status: Important for user engagement

- ☐ Basic analytics tracking (Google Analytics or Plausible)

Tool: Plausible/GA4 | Time: 30 minutes | Status: Essential for understanding users

- ☐ Error handling & logging (Sentry or similar)

Tool: Sentry free tier | Time: 1 hour | Status: Critical for catching bugs

- ☐ Basic security measures (SSL, input validation, rate limiting)

Tool: Built into hosting | Time: 1-2 hours | Status: NON-NEGOTIABLE

NICE-TO-HAVE FEATURES (POST-LAUNCH V2.0)

List features that would be great but aren't critical for launch. You'll add these after you have paying customers and their feedback.

1. _____
2. _____
3. _____
4. _____
5. _____

Example v2.0 features for Password Generator:

1. Password strength meter with real-time feedback

2. Save generated passwords to encrypted vault
3. Browser extension for auto-fill
4. Team sharing and collaboration features
5. Password history and breach checking

The 80/20 Rule: Your MVP should deliver 80% of the value with 20% of the features.
Ask: "Would customers pay for this without feature X?" If yes, X is not MVP.

